

Graph Representations (Adjacency Matrix & Adjacency List) -- Data Structures

Definition: Graph representations are memory data structures used to store the vertices and edge relationships of a graph in computer memory. The two primary methods are Adjacency Matrix and Adjacency List.

1. Detailed Technical Explanation

Sample Graph:

```

    (0) ---- (1)
      |  \   |
      |   \  |
      |    \ |
    (3) ---- (2)

```

1. Adjacency Matrix Representation

A 2D array `adj[V][V]` of size $V \times V$, where `adj[i][j] = 1` if an edge exists from vertex `i` to vertex `j`, and 0 otherwise.

Adjacency Matrix for Sample Graph:

* Pros: Fast edge existence query in $O(1)$ time (`adj[u][v] == 1`).

2. Adjacency List Representation

An array of linked lists `adj[V]`, where each element `adj[i]` points to a linked list of neighboring vertices connected to vertex `i`.

Adjacency List for Sample Graph:

* Pros: Memory efficient $O(V + E)$ for sparse graphs; iterates over neighbors in $O(\text{deg}(u))$ time.

2. Complete Comparison Table

Feature	Adjacency M	Adjacency Li								
**Space Com	** $O(V^2)$ ** (Fi	** $O(V + E)$ ** (								
**Check Edge	$O(1)$ (Instant i	** $O(\text{deg}(u))$ **								
**Find All Nei	** $O(V)$ ** (Sca	$O(\text{deg}(u))$ (Tra								
**Add / Delete	** $O(V^2)$ ** (R	** $O(1)$ ** (App								
**Best Suited	**Dense Grap	E	pprox	V	2 **)	**Sparse Grap	E	V	V	2 **)

3. C Implementation of Adjacency List

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct AdjListNode {
    int dest;
    struct AdjListNode* next;
};

struct Graph {
    int V;
    struct AdjListNode** array;
};

struct Graph* createGraph(int V) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->array = (struct AdjListNode**)malloc(V * sizeof(struct AdjListNode*));
    for (int i = 0; i < V; ++i)
        graph->array[i] = NULL;
    return graph;
}

void addEdge(struct Graph* graph, int src, int dest) {
    // Add edge from src to dest
    struct AdjListNode* newNode = (struct AdjListNode*)malloc(sizeof(struct AdjListNode));
    newNode->dest = dest;
    newNode->next = graph->array[src];
    graph->array[src] = newNode;

    // For undirected graph, add dest to src
    newNode = (struct AdjListNode*)malloc(sizeof(struct AdjListNode));
    newNode->dest = src;
    newNode->next = graph->array[dest];
    graph->array[dest] = newNode;
}
```

4. Must-Write Points for Exams

* In an undirected graph's adjacency matrix, the matrix is always symmetric about the main diagonal ($adj[i][j] == adj[j][i]$).

* Ad

5. Quick Recall Flow

Adj Matrix: 2D Array $V \times V$, $O(V^2)$ Space, $O(1)$ Edge Lookup | Adj List: Array of Linked Lists, $O(V+E)$ Space, Best for Sparse Graphs